LAB 11 - PROMPTS

ML-Signal Stage: a Walk-Forward Classifier from Probability to Position

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab11-ml-signal-classifier/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Build causal features and the label (K4)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/ml.py with build_features(bars): features known at the close of day t only - ret_lag1/2/3/5 (r.shift(lag-1), so ret_lag1 = today's return), mom_5, mom_21, rsi_14 and rsi_2 (pipeline.indicators.ta_rsi), macd_hist / close (ta_macd), Bollinger %B from ta_bb, vol_21 annualised, dist_sma50 and dist_sma200, volume z-score over 21 days; label target = 1 if close t+1 > close t. Drop the last row (no label yet) and rows with NaN. The label may use shift(-1); no FEATURE may. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: build_features(load_bars('SPY','2018-01-01','2025-12-31')) has 1,811 rows and 15 columns (14 features + target).

Step 2: Add walk-forward validation (K4, K6)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/ml.py add MODELS = {'Logistic regression': StandardScaler + LogisticRegression(C=0.1, max_iter=1000), 'Random forest': RandomForestClassifier(n_estimators=300, max_depth=4, min_samples_leaf=20, random_state=42), 'XGBoost': XGBClassifier(n_estimators=200, max_depth=3, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, random_state=42, eval_metric='logloss') imported lazily}, and walk_forward_proba(X, y, make, splits=5) using TimeSeriesSplit(n_splits=5, gap=1): fit on each training block, predict_proba on the following test block, return the out-of-sample probabilities and the last model. Never fit on future data. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: walk_forward_proba returns probabilities only for the five test blocks (2020-01-06 onwards).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Turn probability into a position and backtest it (A4)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/ml.py add position_from_probability(prob, threshold, index=None) = 1.0 where P(up) > threshold (decided at close t, so the backtester fills it at the next open), and run(symbol='SPY', threshold=0.55, models=None) for `python -m pipeline ml --threshold --models`: print samples, features, base rate and the five folds; out-of-sample accuracy, precision, AUC and share predicted up per model, with the naive 'always up' benchmark; backtest each model's positions from the first test date with common.backtest.backtest and the config cost model against buy and hold (CAGR, Sharpe, max drawdown, exposure, trades, turnover); print a VERDICT ('NO EDGE' if the best AUC < 0.53); save _outputs/Lab11_ml_probabilities.csv and chart Lab11_ml_strategy.png (equity curves and XGBoost feature importance from the last fold). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline ml --help shows --threshold and --models.

Step 4: Run the ML stage (A4, K4)

Do this (no prompt needed)
Run: python -m pipeline ml --threshold 0.55 (same as: python3 Lab11-ml-signal-classifier/ml_stage.py).

Check: 1,811 samples, 14 features, base rate 55.4%; folds test from 2020-01-06 to 2025-12-30. Out-of-sample AUC: logistic regression 0.481, random forest 0.483, XGBoost 0.497 (accuracy 0.535 / 0.542 / 0.519 versus 0.551 for always 'up').

Step 5: Read the strategy result (A4)

Do this (no prompt needed)
Read the backtest table and the VERDICT.

Check: 2020-2025, threshold 0.55: CAGR 9.19% / 7.70% / 4.00% versus buy and hold 14.98%; Sharpe 0.61 / 0.54 / 0.32 versus 0.78; turnover 83 / 75 / 96 versus 0.17; 'VERDICT: NO EDGE - keep the ML signal out of the live strategy'.

Step 6: Test the threshold - and notice the snooping risk (A4, K6)

Do this (no prompt needed)
Run: python -m pipeline ml --threshold 0.5 --models 'Logistic regression', then --threshold 0.6.

Check: Threshold 0.50: CAGR 16.82%, Sharpe 0.93, exposure 87%, 128 trades - better than buy and hold; threshold 0.60: 7.67%, Sharpe 0.60, exposure 23%, 205 trades. Choosing 0.50 AFTER seeing this table is data snooping: it must be re-tested out of sample and counted as an extra trial.

Step 7: Prove that leakage is detectable (K6)

Prompt - copy into your AI coding assistant
In a Python shell started in labs, import pipeline.ml, wrap build_features so it also adds a feature 'leak' = tomorrow's return (bars['close'].pct_change().shift(-1)), run walk_forward_proba with the 'Logistic regression' model and print the out-of-sample AUC. Explain why the result proves the feature leaks the future, then discard the code.

Open in Claude | **Open in ChatGPT**

Check: The leaky model scores AUC 0.997 - far too good; the honest model scores 0.481. Accuracy above about 60% on daily direction is a leakage alarm.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Inspect the feature importance (K4)

Do this (no prompt needed)
Open _outputs/Lab11_ml_strategy.png. Which features does XGBoost rely on, and does that make economic sense?

Check: The importance bars are spread across many features with no dominant one - consistent with no reliable edge.

Run it

Step	Command / file
Run Lab 11	python3 Lab11-ml-signal-classifier/ml_stage.py (same as: python -m pipeline ml)
The complete code	pipeline/ml.py (reference copy: Lab11-ml-signal-classifier/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.